

B.TECH. (CSE & IT) 5th Semester Examination 2025

Software Engineering (BCS-601) - Test Solutions

Detailed Solution Report

1 Cyclomatic Complexity in Basic Path Testing

Cyclomatic complexity is a software metric used to indicate the complexity of a program. It is a quantitative measure of the number of linearly independent paths through a program's source code.

1.1 Calculation Methods

There are three ways to calculate the cyclomatic complexity $V(G)$ for a flow graph G :

1. **Based on Regions:** $V(G) = \text{Number of enclosed regions} + 1$.
2. **Based on Edges and Nodes:** $V(G) = E - N + 2P$, where E is the number of edges, N is the number of nodes, and P is the number of connected components (usually $P = 1$ for a single procedure).
3. **Based on Predicate Nodes:** $V(G) = P + 1$, where P is the number of predicate nodes (nodes containing conditions).

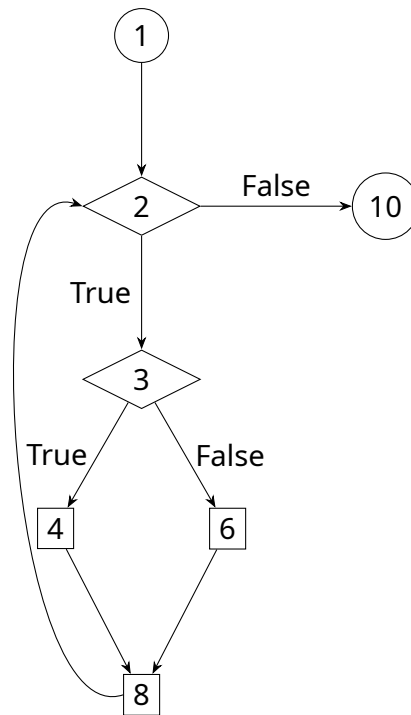
1.2 Example Procedure and Calculation

Consider the following pseudocode for a simple procedure that finds the maximum of two numbers within a loop:

```
1. Start
2. While (a > 0)
3.   If (a > b)
4.     print a
5.   Else
6.     print b
7.   End If
8.   a--
9. End While
10. Stop
```

1.2.1 Control Flow Graph (CFG)

The diagram below represents the logical flow of the procedure.



1.2.2 Calculations

- **Nodes (N):** 7 (Nodes 1, 2, 3, 4, 6, 8, 10)
- **Edges (E):** 8
- **Predicate Nodes (P):** 2 (Nodes 2 and 3)

Using Formula 2: $V(G) = E - N + 2 = 8 - 7 + 2 = 3$.

Using Formula 3: $V(G) = P + 1 = 2 + 1 = 3$.

The cyclomatic complexity is **3**, indicating there are 3 independent paths to test.

2 Software Design Paradigms

2.1 Function Oriented Design

This approach views a system as a collection of interacting functions. The system state is centralized and shared among various functions.

- **Example:** A Payroll System where different functions like `calculate_salary()`, `generate_tax_report()` and `update_attendance()` operate on a central employee database.

2.2 Top-down and Bottom-up Design

- **Top-down Design:** Starts with a high-level description of the system and iteratively decomposes it into smaller, more manageable sub-modules.
 - *Example:* Designing a car by starting with the "Vehicle" concept, then dividing it into "Engine," "Chassis," and "Electronics."
- **Bottom-up Design:** Starts with the lowest-level components (basic building blocks) and combines them to create higher-level abstractions.
 - *Example:* Building a library of reusable UI components (buttons, text fields) first, then assembling them to create a "Login Page."

2.3 Object Oriented Design (OOD)

OOD views the system as a collection of objects that interact by passing messages. Each object encapsulates data (attributes) and behavior (methods).

- **Example:** In a Banking Application, `Account` is a class. Each user has an `Object` of that class. Methods like `withdraw()` or `deposit()` modify the object's internal state.

3 Halstead's Software Science

Maurice Halstead's metrics are based on the count of operators and operands in a program.

3.1 Fundamental Metrics

- n_1 : Number of distinct operators.
- n_2 : Number of distinct operands.
- N_1 : Total number of operator occurrences.
- N_2 : Total number of operand occurrences.

3.2 Derived Metrics

- **Program Vocabulary:** $n = n_1 + n_2$
- **Program Length:** $N = N_1 + N_2$
- **Volume (V):** $V = N \times \log_2(n)$ (Size of the program in bits)
- **Difficulty (D):** $D = \frac{n_1}{2} \times \frac{N_2}{n_2}$
- **Effort (E):** $E = D \times V$

3.3 Example

Code snippet: `result = a + b * 2;`

- Operators: `=`, `+`, `*` ($n_1 = 3$)
- Operands: `result`, `a`, `b`, `2` ($n_2 = 4$)
- If total occurrences are $N_1 = 3$ and $N_2 = 4$, then $n = 7, N = 7$.
- Volume $V = 7 \times \log_2(7) \approx 7 \times 2.8 = 19.6$ bits.

4 Code Walkthrough and Review

4.1 Code Walkthrough

A code walkthrough is an informal technical review where the author of the code leads a team of peers through the logic of the code. The primary goal is to find bugs and improve understanding, not to evaluate the developer.

4.2 Important Rules for Walkthrough and Review

1. **Review the code, not the person:** Focus on technical flaws rather than criticizing the developer's style or intelligence.
2. **Pre-work is essential:** Reviewers should read the code before the meeting to identify potential issues.
3. **Limit the session:** Meetings should generally last no more than 60–90 minutes to prevent "reviewer fatigue."
4. **Use Checklists:** Use standard checklists for common errors (memory leaks, boundary conditions, security vulnerabilities).
5. **Small Team Size:** Ideally, 3–5 members including the moderator and recorder.

5 Compliance and Testing

5.1 Compliance to Coding Standards

Compliance is checked during:

- **Coding Phase:** Using automated "Linters" or static analysis tools.
- **Code Review Phase:** Peers manually verify if naming conventions, indentation, and architectural patterns are followed.

5.2 Alpha vs Beta Testing

Table 1: Comparison of Alpha and Beta Testing

Feature	Alpha Testing	Beta Testing
Location	Internal (Developer's site)	External (User's site)
Testers	Internal employees/QA team	Real end-users/Customers
Environment	Controlled environment	Real-world/Live environment
Timing	Performed before Beta testing	Performed before final release
Reliability	Focus on functionality	Focus on usability and reliability